# Modeling, Identification, and Control of a Ball Stabilization System: A practical case study

**Group 11**

Alexis Santucci (2209551)

Costin Stanicel (1824589)

Thomas Roose (2055163)

Simon Rosenberg (1844636)

June 28, 2026

Eindhoven University of Technology

***Abstract*— In this paper, multiple advanced control techniques, including optimal control, $H_\infty$ syntheses, and predictive control models, are applied to a ball and plate setup. This design includes physical modeling, system identification, simulation, and real-time implementation. Loop shaping techniques are applied to the inner loop control of the plate actuators to ensure a good position of the plate. A 3D camera is used to measure the position of the ball on the plate, and its relative height, and the outer loop controllers use this measurement to command an angle to the plate and thereby move the ball. The performance of 4 different outer loop controllers is compared in terms of error and control effort used during a square reference tracking, and a simple proportional controller is applied to track a reference in the vertical direction.**

## I. Introduction

Most control-related research papers focus on the theoretical proofs and simulations, and generally have small experimental implementation sections. Furthermore, most papers have a narrow focus on a specific control algorithm. Thus, there is a significant gap between theoretical control strategies and the actual implementation on an experimental setup. To address this gap, this paper presents the control of a ball-and-plate setup. The ball-and-plate system is an interesting engineering multi-axis system, consisting of a dual-control loop. One is for controlling the height and angle of the plate, using 3 linear actuators, and the other is for controlling the ball position on the plate.

This paper comprehensively details the design, modeling, and implementation steps required to control the dual-loop. It starts with the modeling of the system, which can be later used for controller design. The controller design is tested in simulation and on hardware. Furthermore, a 3D camera is implemented to read out the ball position, which is integrated into the control loop.

## II. Modeling

The ball and plate system consists of a plate that is lifted up by three linear actuators and ball that is free to move on top of the plate.

### A. Kinematics

In order to define a first-principles model of the system, we must first define the kinematics of the system. Having adequate mathematical models is essential for understanding and controlling the dynamics of the ball-and-plate system.

Fig. 1 shows the kinematics of the actuating point $A$ on the plate. Here, the fixed world origin $O$ is the base of the actuators directly below the middle of the plate. The middle of the plate is the point $P$, which is also the origin of the relative frame $O'$. This frame is rotated with respect to (w.r.t) the real world by the Tait-Bryan rotation matrix.

$$\underline{A}^{0'0} = \begin{bmatrix} C_\psi C_\beta & C_\psi S_\beta S_\alpha - S_\psi C_\alpha & C_\psi S_\beta C_\alpha + S_\psi S_\alpha \\ S_\psi C_\beta & S_\psi S_\beta S_\alpha + C_\psi C_\alpha & S_\psi S_\beta C_\alpha - C_\psi S_\alpha \\ -S_\beta & C_\beta S_\alpha & C_\beta C_\alpha \end{bmatrix} \tag{1}$$

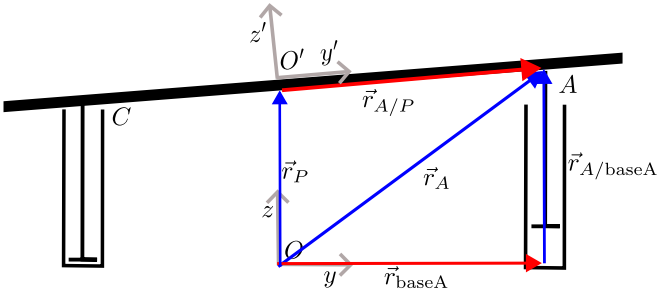


Fig. 1: Side view of plate system, showing position vectors of actuating point $A$. Here, constant vectors are red and time-varying vectors are blue.

Where $C_j = \cos(j)$, and $S_i = \sin(i)$. Given that the plate is fixed around the $z$ axis, we have $\psi = 0$. So, the rotation matrix simplifies to

$$\underline{A}^{0'0} = \begin{bmatrix} C_\beta & S_\beta S_\alpha & S_\beta C_\alpha \\ 0 & C_\alpha & -S_\alpha \\ -S_\beta & C_\beta S_\alpha & C_\beta C_\alpha \end{bmatrix}. \tag{2}$$

The known vectors are defined as

$$\begin{aligned} \vec{r}_P &= (\underline{r}_P^0)^\top \underline{\vec{e}}^0 = \begin{bmatrix} 0 & 0 & h \end{bmatrix} \underline{\vec{e}}^0 \\ \vec{r}_{\text{baseA}} &= (\underline{r}_{\text{baseA}}^0)^\top \underline{\vec{e}}^0 = \begin{bmatrix} 0 & r & 0 \end{bmatrix} \underline{\vec{e}}^0 \\ \vec{r}_{A/P} &= (\underline{r}_{A/P}^{0'})^\top \underline{\vec{e}}^{0'} = \begin{bmatrix} 0 & r & 0 \end{bmatrix} \underline{\vec{e}}^{o'} = \begin{bmatrix} 0 & r & 0 \end{bmatrix} \underline{A}^{0'0} \underline{\vec{e}}^0 \end{aligned} \tag{3}$$

With $h = 0.32$ m representing the height from the base to the plate and $r = 0.17$ m representing the radius from the base origin to the actuators. The movement of the plate w.r.t to the base is possible due to the flexible fixture of each actuator. This allows a small deviation of the vertical z-axis. Using these vectors, we can define

$$\vec{r}_A = \vec{r}_P + \vec{r}_{A/P} \tag{4}$$

and

$$\vec{r}_A = \vec{r}_{\text{baseA}} + \vec{r}_{A/\text{baseA}} \tag{5}$$

By setting (4) equal to (5), we get

$$\begin{aligned} \vec{r}_{A/\text{baseA}} &= \vec{r}_P + \vec{r}_{A/P} - \vec{r}_{\text{baseA}} \\ (\underline{\vec{r}}_{A/\text{baseA}}^0)^\top \underline{\vec{e}}^0 &= \left( \underline{r}_P^0 + \underline{r}_{A/P}^{0'} \underline{A}^{0'0} - \underline{r}_{\text{baseA}}^0 \right)^\top \underline{\vec{e}}^0 \end{aligned} \tag{6}$$

Then, the total length of the actuator is given by the norm of this vector. To find the total movement required by the actuator, the distance from the base A to the midpoint of the actuator's travel is subtracted.

$$p_A = ||\vec{r}_{A/\text{baseA}}|| - h \tag{7}$$

The same calculation is applied to compute the required setpoint for actuators B and C. The layout of these actuators is given in Fig. 2. Here the distances from the base origin to the actuator base are given by

$$\begin{aligned} \vec{r}_{\text{baseB}} &= (\underline{r}_{\text{baseB}}^0)^\top \underline{\vec{e}}^0 = \begin{bmatrix} -r\cos(\frac{\pi}{6}) & r\sin(-\frac{\pi}{6}) & 0 \end{bmatrix} \underline{\vec{e}}^0 \\ \vec{r}_{\text{baseC}} &= (\underline{r}_{\text{baseC}}^0)^\top \underline{\vec{e}}^0 = \begin{bmatrix} r\cos(-\frac{\pi}{6}) & r\sin(-\frac{\pi}{6}) & 0 \end{bmatrix} \underline{\vec{e}}^0. \end{aligned} \tag{8}$$

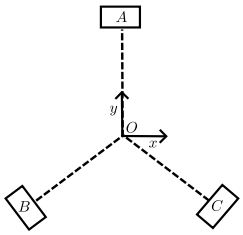


Fig. 2: Top-down view of actuator layout.

Using these computations, the inverse kinematics, from the desired angle to the actuator position, are known. In order to have angle feedback in the control loop, the backward kinematics, from position to angles, are also needed. To make this computation simpler, an assumption must be made. While the above computation computes the norm of the vector $\vec{r}_{A/\text{baseA}}$, if we assume the deviation in the $x$ and $y$ directions is zero, this is equivalent to extracting the $z$ component. Using this, the vertical height of the actuating points can be defined as

$$\begin{aligned} z_A &= h + r\cos(\beta)\sin(\beta) \\ z_B &= h + r\frac{\sqrt{3}}{2}\sin(\beta) - r\frac{1}{2}\cos(\beta)\sin(\alpha) \\ z_C &= h - r\frac{\sqrt{3}}{2}\sin(\beta) - r\frac{1}{2}\cos(\beta)\sin(\alpha). \end{aligned} \tag{9}$$

These three equation can be solved for the angles

$$\begin{aligned} \beta &= \sin^{-1}\left(\frac{z_B - z_C}{r\sqrt{3}}\right) \\ \alpha &= \sin^{-1}\left(\frac{z_A - h}{r\cos(\beta)}\right). \end{aligned} \tag{10}$$

### B. Plate dynamics

Fig. 3 shows a simplified model of one of the actuators. Here, the total mass each actuator has to move is defined as the mass of the actuator piston and one-third of the mass of the plate.

$$m_T = m_A + \frac{1}{3}m_p \tag{11}$$

In this model, the forces acting on the mass are simply gravity pulling it down, the viscous friction force acting opposite to the movement direction, and the actuating force itself.

To derive the equations of motion, Newton's second law is applied.

$$\begin{aligned} \sum F &= m_T\ddot{z}(t) \\ -G + F_m - F_v &= m_T\ddot{z}(t) \\ m_T\ddot{z}(t) &= K_T I(t) - C_v\dot{z}(t) - m_T g \end{aligned} \tag{12}$$

Where $K_T = 11$ N/A is the motor constant and $C_v = 16.5$ Ns/m is the viscous damping coefficient [3].

In order to derive the transfer function, the system is analyzed using deviation variables around its steady-state operating point. In this position, velocity and acceleration

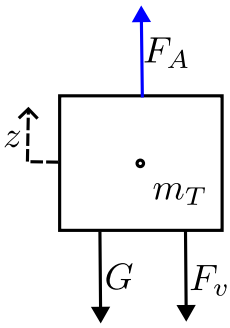


Fig. 3: Free body diagram of the actuator, modeled as a point mass with forces acting on it in the $z$ direction.

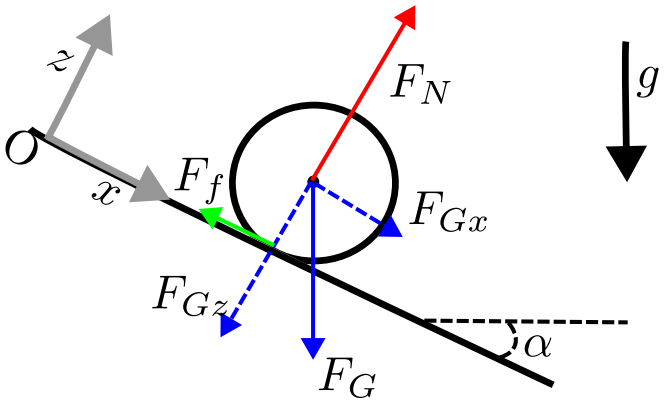


Fig. 4: Free body diagram of ball

are zero, and the constant holding current ($I_{eq}$) perfectly counteracts the force of gravity ($K_T I_{eq} = m_T g$). By defining the variables as deviations from this equilibrium ($I(t) = I_{eq} + \Delta I(t)$ and $z(t) = z_{eq} + \Delta z(t)$), the constant gravity term cancels out of the dynamic equation:

$$m_T\Delta\ddot{z}(t) = K_T\Delta I(t) - C_v\Delta\dot{z}(t) \tag{13}$$

Now, a Laplace transformation can be applied to find the transfer function from input current $I(s)$ to the position of the actuator mass $z(s)$.

$$\frac{z(s)}{I(s)} = \frac{\frac{K_T}{m_T}}{s(s + \frac{C_v}{m_T})} \tag{14}$$

### C. Ball dynamics

In order to model the dynamics of the ball, first-principle modelling is applied to set up a state space model. To do this, we must first define the system itself. The free body diagram of the ball is given in Fig. 4. This shows the ball's movement over the $(x, z)$ plane, which is controlled by the $\alpha$ angle. The same dynamics apply to the movement over the $(y, z)$ plane, which is influenced by the $\beta$ angle. We make the following assumptions of this system.

- The ball does not slip with respect to the plate.
- The ball cannot bounce on the plate.
- The ball lies on an infinite plane; there are no walls.
- The angles of the plate $\alpha$ and $\beta$ are small enough to allow linearization.

- The $x$ and $y$ axes are decoupled, i.e. a change in the angle $\alpha$ only affects the balls $x$ position and a change in the balls $\beta$ position only affects its $y$ position.

Using Newton's second law, we define

$$m\ddot{x} = mg\sin(\alpha) - F_j \tag{15}$$

Where the first term is the component of the gravity that is parallel to the plate. Furthermore, Euler's equation of angular momentum gives

$$\begin{aligned} \tau &= I\dot{\omega} \\ F_j r &= I\dot{\omega} \\ F_j r &= \frac{2}{5}mr^2\frac{\ddot{x}}{r} \\ F_j &= \frac{2}{5}m\ddot{x} \end{aligned} \tag{16}$$

Now substitute (16) into (15) to get

$$\begin{aligned} m\ddot{x} &= mg\sin(\alpha) - \frac{2}{5}m\ddot{x} \\ \ddot{x} &= \frac{5}{7}g\sin(\alpha). \end{aligned} \tag{17}$$

Given that the plate will not exceed angles larger that 10°, this equation can be linearized to

$$\ddot{x} = k_m\alpha. \tag{18}$$

Where $k_m = \frac{5}{7}g$ is the rolling constant. Similarly the acceleration in the $y$ direction can be derived as,

$$\ddot{y} = -k_m\beta. \tag{19}$$

This can be written into state space form,

$$\begin{aligned} \dot{x} &= \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ k_m & 0 \\ 0 & 0 \\ 0 & -k_m \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \\ y(k) &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix} \end{aligned} \tag{20}$$

This model is completely controllable and observable. That is, the rank of the controllability matrix $\mathcal{C}$ and the rank of the observability matrix $\mathcal{O}$ equals the state space dimension $n = 4$. It is a marginally stable system, which means that any small acceleration applied to the ball will cause the ball to move away from its equilibrium at a constant speed until more acceleration is applied.

### *D. System identification*

The first-principles model of the plate and the 3 linear actuators is an adequate first step toward obtaining a mathematical model. However, system identification can be used to obtain a more accurate mathematical model of the plate system. It does so using experimental data.
Accurate models are useful for designing controllers for each actuator.

The plate system model should be identified at its operating point. This point lies midway through the possible stroke of the actuators. Therefore, a feedback controller is needed to ensure the plate can be lifted and stays at its operating position.
The first-principles model is therefore used to design a conservative feedback controller.
The identification measurements can now be performed using the conservative feedback controller. The identification experiments are performed using the following steps:

1) Lift the plate upwards to its operating point
2) Apply a uniform random signal at motor A, and measure the currents and positions of each actuator.
3) Repeat for motor B and C
4) Lower the plate back down when finished measuring

The excitation signal is a discrete uniform random sequence,

$$r(n) \sim \mathcal{U}(a, b). \tag{21}$$

Where $a = -1.8$ A and $b = 1.8$ A are the lower and upper bounds of the signal amplitude, respectively. This ensures that the applied current does not exceed the saturation, while persistently exciting the system across its frequency spectrum. Table I gives the parameters used for signal generation.

TABLE I: Signal Parameters Used in the Experiment

| Parameter | Value |
|---|---|
| $N$ | 200000 |
| $f_s$ | 1000 Hz |
| $\|A\|$ | 1.8 A |

First, a frequency-response-function (FRF) of the transfer from current to height is obtained using the data from the 3 experiments. These FRFs are obtained using the 3-point method,

$$H(f) \approx -\frac{\sum_{i=1}^{N} S^i_{-\text{ed}}(f)}{\sum_{i=1}^{N} S^i_{\text{ud}}(f)} \tag{22}$$

where $S$ is the cross power spectral density, which is averaged to approximate the plant, according to Eq. (22). This is used as a first approximation to obtain insights into the system. These FRFs can now be used to assess the interaction between different channels, using the relative gain array (RGA) defined as:

$$\Lambda(\omega j) = G \odot (G^{-1})^\top \tag{23}$$

Where $\odot$ is an element-wise multiplication. The RGA is a measure of how much interaction is present in the system. The system is perfectly decoupled if the RGA shows a perfect identity transfer matrix.

The RGA presented in Fig. 5 shows that the system is decoupled. Only small peaks appear toward higher frequencies, which might also be explained by the poor signal-to-noise ratio (SNR).

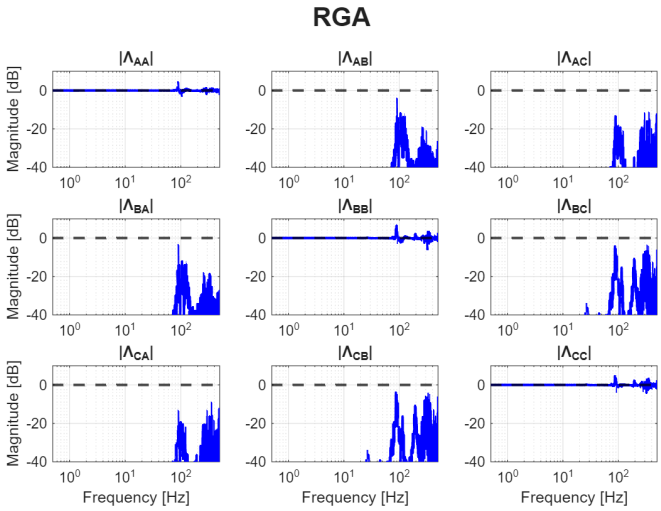


Fig. 5: Relative Gain Array of the ball-and-plate setup.

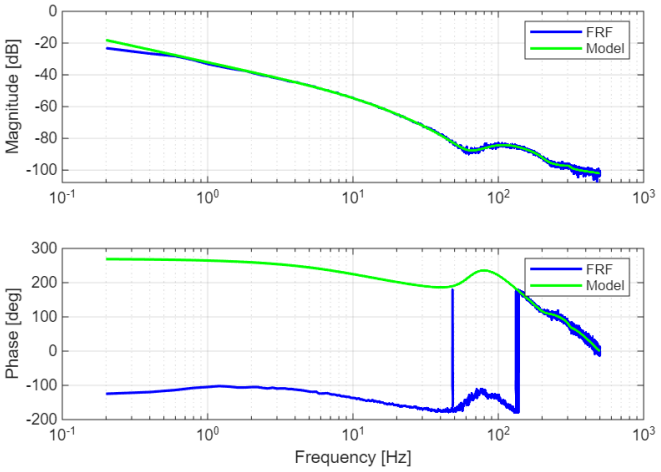


Fig. 7: Bode plot of motor A of the identified model and FRF.

A decoupled system means that the models can be derived for each motor, like in a (single-input-single-output) SISO setting. Furthermore, controllers can also be designed per motor.

The parametric models for motor A, B, and C can be obtained using closed-loop identification techniques. The direct time-domain approach is considered, which uses predication error minimization techniques. [4] Here, a consistent estimate can be obtained when $\mathcal{S} \in \mathcal{M}$ and having a persistently exciting input. Where $\mathcal{M}$ contains both the full model and noise model $(\mathcal{G}, \mathcal{H})$. This means that a Box-Jenkins model is needed to fully capture both the noise model and the plant dynamics $G$.
The input for time-domain estimation is now chosen to be a uniform random signal, with an amplitude of 1.8 A. This uniform random signal is of a persistently exciting order going towards infinity. Therefore, making it an excellent choice for time-domain estimation.

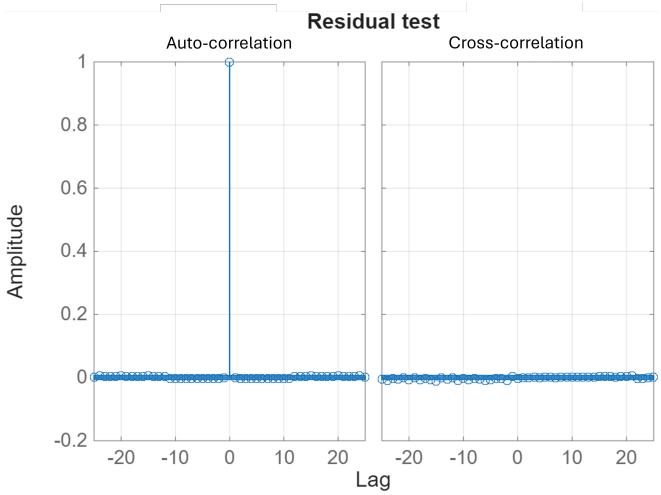


Fig. 6: Residuals test for motor A.

The residual validation test is an easy-to-interpret method to assess whether the model fully describes the dynamics. [4] However, both the cross- and auto-correlation of the predication error have to be interpreted simultaneously for closed-loop direct identification. The autocorrelation is a test to verify whether the dynamics are captured. The cross-correlation is used to verify whether the noise model is correct. The residual test for the final model of motor A is presented in Fig. 6.

All the motors are identified using a Box-Jenkins model to obtain a plant estimate $\hat{G}$ and the noise-model $\hat{H}$. The resulting model and FRF of motor A are shown in Fig. 7. The parametric Box-Jenkins model and non-parametric FRF nicely overlap. The models and FRFs are obtained in the same way as motor A, and are presented in Appendix B.

The system starts with a single integrator at the lower frequencies, indicated by the -1 slope. This is logical for an electromechanical system, since the system is dominated by back-EMF or viscous damping at the lower frequencies, which is related to velocity. Thus, a single integrator is present, since we measure position. Next, the system transitions to a double integrator, as the damping forces become negligible compared to the inertial forces, which are directly linked to acceleration. The anti-resonance and resonance show that there is a compliant connection between 2 masses.

## III. Inner Loop Controller Design

In order to accurately control the plate to any angle within its range, controllers must be designed for each linear actuator. These controllers should be able to respond to requested angles fast enough (high bandwidth) while ensuring zero steady state error and stability. The inner loop control input is defined as the requested angles

$$u = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \tag{24}$$

These angles are converted to actuator setpoints using the inverse kinematics defined in section II. These positions are then used as the setpoint for each actuator.

TABLE II: Controllers' parameters for actuator A

| Controller Block | Parameters |
|---|---|
| Proportional Gain | $K_p = 500$ |
| Integrator | $f_I = 1$ Hz |
| Lead filter | $f_z = 7.5$ Hz, $f_p = 30$ Hz |
| 1st order lowpass Filter | $f_c = 50$ Hz |

### A. Loop shaping

Loop shaping is applied to design a controller for each actuator. Here, the parametric models defined in section II are used, and different controller blocks are applied until the desired closed-loop performance and robustness are reached. Table II gives the parameters used to design the controller in ShapeIt [1]. This controller contains a lead filter, which adds phase around the bandwidth frequency by placing a pole before it and a zero after it. In this case, roughly half of the desired bandwidth is used for the zero $f_z$ and roughly double the desired bandwidth for the pole $f_p$. An integrator is added to ensure a near-zero steady-state error. A first-order lowpass filter is added at a frequency well above the desired bandwidth to attenuate high-frequency noise. And finally, a proportional gain is applied to tune the bandwidth.

### B. Closed loop stability and robustness

The closed stability of this controller is analyzed using the Nyquist criterion. Given that the plant itself has no unstable poles, the Nyquist contour should not encircle the -1 point. Fig. 8 shows the Nyquist contour, which clearly does not encircle the -1 point, which implies closed-loop stability.

The same controller structure is applied to the other two actuators, where the proportional gain is retuned to compensate for the slight variation in actuator dynamics. This results in good robustness margins and a bandwidth around 14 Hz, as shown in Table III.

### C. Integrator implementation

In order to safely implement the controllers designed above, integrator anti-windup is applied. This ensures the Integrator cannot exceed a saturation limit [8, App. F].

$$\text{sat}(u) = \begin{cases} u_{\text{sat}} & \text{if } u > u_{\text{sat}} \\ u & \text{if } -u_{\text{sat}} > u > u_{\text{sat}} \\ u_{\text{sat}} & \text{if } u < -u_{\text{sat}} \end{cases} \tag{25}$$

Where $u_{\text{sat}} = 1$ A. Furthermore, an integrator reset is applied to allow the current output value of the integrator to be set to zero at any time. This is connected to the controller enable signal in the model to ensure that the integrator is not already at saturation when the controller is enabled.

TABLE III: Controller statistics

| | MM[dB] | PM[°] | GM[dB] | $f_{BW}$[Hz] |
|---|---|---|---|---|
| **Motor A** | 4.89 | 50.2 | inf | 14.20 |
| **Motor B** | 4.46 | 48.2 | inf | 14.71 |
| **Motor C** | 5.57 | 39.4 | 10.2 | 14.40 |

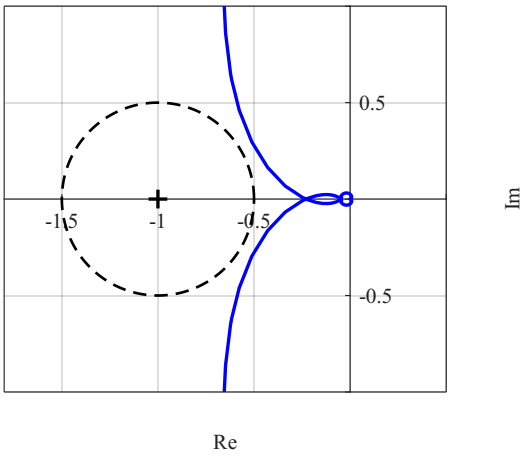


Fig. 8: Nyquist plot of controller $A$.

Both of these functions are implemented in Matlab Simulink using the built-in discrete integrator block. This block takes a gain $K_I$ as input, which can be computed by

$$K_I = \frac{2\pi f_I}{K_{\text{DC}}} \tag{26}$$

Where $f_I$ is the integrator cutoff frequency designed in Shapeit and $K_{\text{DC}}$ is the DC gain of the controller without any integrator.

### D. Step response

In order to validate the actuator models and controller performance, a step response is simulated and then compared to a step response on the experimental setup. Given that the controller is known, if these two-step responses are the same, this implies the model is close to the true system. A step size of 1 cm is chosen because this is well within the range of motion of the system. From Fig. 9, it is clear that the simulated and experimental step responses have similar shapes. However, the experimental results have a longer delay. This is also seen from the longer rise time in Table IV.

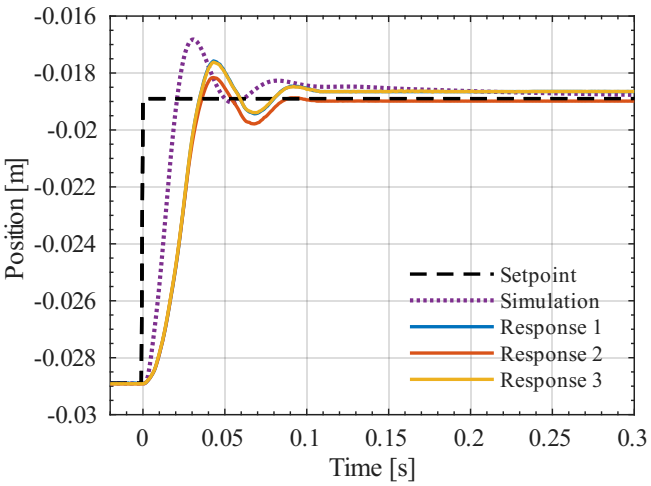


Fig. 9: Comparison between simulated and experimental step responses on motor A.

This likely means there are some delays in the true system that have not been modeled accurately. Moreover, the true bandwidth of the system is most likely lower than the model-based bandwidth, leading to a slightly slower response. From the steady state error, it is clear that the integrator works well and pushes the error to near zero after each movement.

TABLE IV: Step Response Performance Metrics

| **Response** | **OS (%)** | $T_r$ **(s)** | $T_s$ **(s)** | $e_{ss}$ **(m)** |
|---|---|---|---|---|
| Simulation | 20.80 | 0.0138 | 0.2425 | $8.24 \times 10^{-9}$ |
| Experiment 1 | 13.24 | 0.0213 | 0.3692 | $1.80 \times 10^{-5}$ |
| Experiment 2 | 7.39 | 0.0221 | 0.0825 | $4.00 \times 10^{-6}$ |
| Experiment 3 | 12.80 | 0.0214 | 0.3684 | $1.80 \times 10^{-5}$ |

OS: Overshoot, $T_r$: Rise Time, $T_s$: Settling Time, $e_{ss}$: Steady-State Error.

## IV. Observer Design

The ball dynamics are given by Eq. (20), where only the position of the ball is measured. Therefore, the ball's velocity is still unknown, which is, however, essential for state-feedback controllers.
Consequently, designing an observer that can give accurate full state estimates is crucial. To this end, a Kalman filter is designed to estimate the state vector and handle noisy camera measurements. The key idea is to combine a prediction model based on the ball dynamics in Eq. (20) with the measurements from the camera $y_k$, and weighing them based on their uncertainty.

### A. Kalman filter calculations

Consider a general state-space system given by

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + w_k \\ y_k &= Cx_k + n_k. \end{aligned} \tag{27}$$

The signal $n_k$ represents the measurement noise, and $w_k$ is the process noise. The process noise captures model uncertainties, such as unmodeled dynamics, disturbances, and discretization errors. The signals $n_k \sim \mathcal{N}(0, V)$ and $w_k \sim \mathcal{N}(0, W)$ are zero-mean Gaussian processes with covariance matrices $V$ and $W$, respectively.
The goal is to obtain accurate estimates for the full state vector $\hat{x}$. The Kalman filter provides estimates by performing 2 calculations at each time step. First, a prediction is made using the model, then a correction is executed using the measurement data.
The prediction step provides a prediction of what the state would be in Eq. (28) based on the model and the previous estimate $\hat{x}_{k|k}$ and input $u_k$.

$$\hat{x}_{k+1|k} = A\hat{x}_{k|k} + Bu_k \tag{28}$$

Furthermore, the uncertainty of the predicted state $\hat{x}_{k+1|k}$ is increased using Eq. (29) through the process noise covariance matrix $W$.

$$\Phi_{k+1|k} = A\Phi_{k|k}A^\top + W \tag{29}$$

The second step is to perform a correction on the state estimate based on Eq. (28) and Eq. (29). First, the Kalman gain $L_k$ is updated in Eq. (30), using the transition matrix $\Phi_{k|k-1}$ from Eq. (29).

$$L_k = \Phi_{k|k-1}C^\top(C\Phi_{k|k-1}C^\top + V)^{-1} \tag{30}$$

Consequently, the state estimate $\hat{x}_{k|k}$ for the current time step is calculated using Eq. (31).

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + L_k(y_k - C\hat{x}_{k|k-1}) \tag{31}$$

Here, $y_k - C\hat{x}_{k|k-1}$ is called the innovation, it represents the mismatch between the actual measurement and the predicted output. The Kalman gain $L_k$ determines how strongly this mismatch influences the corrected estimate.
Lastly, the corrected transition matrix $\Phi_{k|k}$ is calculated using Eq. (29).

$$\Phi_{k|k} = \Phi_{k|k-1} - \Phi_{k|k-1}C^\top(C\Phi_{k|k-1}C^\top + V)^{-1}C\Phi_{k|k-1} \tag{32}$$

The cycle with prediction and correction happens each time step. The state estimate $\hat{x}_{k|k}$ is then used as input for state-feedback controllers.

### B. Selecting covariance matrices

The filter uses the covariance matrices $V$ and $W$ to update its estimates. However, finding these matrices to obtain usable estimates is challenging.
The measurement noise covariance matrix $V$ describes the uncertainty of the camera measurements. A larger $V$ means the measurements are considered unreliable, causing the filter to trust the model prediction more.
The matrix $W$ captures uncertainty in the model, as discussed earlier. Here, a larger $W$ means to rely more on the measurements. Individual states can be weighted differently in $W$, which is often needed when estimating position and velocity. The velocity is often poorly modeled and therefore has larger weights in $W$ compared to positions.
The effects of covariance matrices are summarized in Table V.

The Kalman observer, after tuning, can be used for the state controllers presented in the next section. The tuned parameters for the Kalman filter are given by

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 100 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 100 \end{bmatrix} \quad V = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}. \tag{33}$$

TABLE V: Effect of Kalman filter covariance matrices

| Parameter | Effect Summary |
|---|---|
| $V$ | ↑: trust model more, smoother estimates<br>↓: trust measurements more, noisier estimates |
| $W$ | ↑: trust measurements more, faster correction<br>↓: trust model more, slower correction |

## V. OUTER LOOP CONTROLLER DESIGN

Now that the plate angle can be controlled accurately with the inner-loop controllers defined in Section III and the full state is known using the observer explained in IV, the outer-loop architecture can be defined. Four different feedback control approaches are presented below; each is designed to control the ball's position by adjusting the plate's angle. Therefore, the plant input is defined as

$$u = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}. \tag{34}$$

where $\alpha$ and $\beta$ are the commanded plate angles in radians. Furthermore, a simple feedforward controller is developed to enhance reference tracking.

### A. Feedforward control

Developing a feedforward controller could potentially lead to enhanced tracking performance. A mass-feedforward controller is designed using the model in Eq. (20). The accelerations for positions $x$ and $y$ are given by Eq. (18) and Eq. (19), respectively.
Consider a state reference $r$, this reference has a certain required acceleration $\ddot{r}_x$ and $\ddot{r}_y$. An inverse model can be obtained from the model in Eq. (20). This results in

$$u_{ff} = \begin{bmatrix} \alpha_{ff} \\ \beta_{ff} \end{bmatrix} = \begin{bmatrix} \frac{7}{5g}\ddot{r}_x \\ -\frac{7}{5g}\ddot{r}_y \end{bmatrix} \tag{35}$$

which can be added to the control action from feedback, where the total angular input is now given as

$$u_{\text{total}} = u_{ff} + u_{feedback}. \tag{36}$$

### B. Traditional Feedback Control - PID

A proportional-integral-derivative (PID) controller is selected as the initial control strategy due to its widespread use in industrial applications and its relatively simple implementation.
Although more advanced control techniques are investigated throughout this project, the PID controller provides a useful performance benchmark against which the remaining controllers can be evaluated.

The controller is implemented using the Simulink PID Controller block, which is defined as:

$$P + I.T_s\frac{1}{z-1} + D\frac{N}{1 + N.T_s\frac{1}{z-1}} \tag{37}$$

Independent PID controllers were used for the x- and y-directions, exploiting the approximately decoupled dynamics of the two axes around the operating point.
The proportional term $P$ generates a control action proportional to the position error, the integral term $I$ removes steady-state error, while the derivative term $D$ improves damping by reacting to the rate of change of the error. To reduce the amplification of measurement noise, the derivative action is filtered using the built-in first-order derivative filter of the Simulink PID block.

Several approaches were considered for tuning the controller parameters. A model-based tuning approach using classical loop-shaping techniques is first considered. However, the simplified ball-and-plate model neglects several practical effects present in the experimental setup, including friction, camera latency, and measurement noise. Consequently, gains obtained solely from the model were not expected to provide optimal experimental performance.
A second possibility consisted of tuning the controller using the Simulink model. While this approach provided useful insight into the expected behaviour, its effectiveness remained limited by the speed of the simulation.

The final tuning method is therefore performed mostly experimentally using the startup values found using the simulation. The reference position is fixed at the centre of the plate, corresponding to $(x, y) = (0, 0)$, while the ball is manually displaced to introduce disturbances.
The controller response is observed, and the proportional, integral and derivative gains were adjusted iteratively using the rule of thumb until a satisfactory compromise between response speed, overshoot and disturbance rejection is obtained.
This approach proved to be the most effective, as it directly accounted for the dynamics of the physical system that were not captured by the simplified model.

The final controller parameters are given in Table VI.

### C. $H_\infty$ synthesis

The optimal controller is designed as an outer loop controller for the ball and plate setup. The purpose of this controller is to regulate the ball position while accounting for tracking performance, actuator limitations, disturbance rejection, and high-frequency noise amplification. The optimal controller is synthesized offline using an $H_\infty$ formulation.

For the synthesis control design, the ball dynamics are described using a four-state model. The state vector is chosen as

$$x_r = \begin{bmatrix} x & \dot{x} & y & \dot{y} \end{bmatrix}^\top, \tag{38}$$

The ball acceleration is assumed to be proportional to the plate angle. Following the convention introduced in the modelling section, this gives

$$\ddot{x} = k_m\alpha, \qquad \ddot{y} = -k_m\beta, \tag{39}$$

with

$$k_m = \frac{5}{7}g. \tag{40}$$

TABLE VI: The final tuned parameters

| Parameter | Numerical value |
|---|---|
| $K_P$ | 0.7 |
| $K_I$ | 0.08 |
| $K_D$ | 0.15 |
| $N$ | 25 |

Using these definitions, the nominal ball model can be written as

$$\dot{x}_r = A_r x_r + B_r u, \tag{41}$$

where

$$u = \begin{bmatrix} \alpha & \beta \end{bmatrix}. \tag{42}$$

The state space matrices are then given by

$$A_r = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \qquad B_r = \begin{bmatrix} 0 & 0 \\ k_m & 0 \\ 0 & 0 \\ 0 & -k_m \end{bmatrix}. \tag{43}$$

The actuator dynamics are taken into account. The actuator model derived in the modelling section is used to describe the relation between the motor input and actuator position,

$$G_{\text{act}}(s) = \frac{K_T/m_T}{s\left(s + \frac{C_v}{m_T}\right)}. \tag{44}$$

This actuator model captures the finite bandwidth of the inner actuator system. The commanded plate angles are first converted to actuator position references using the inverse kinematics. The actuator model then represents the response from these references to the realised plate motion. Including this model is important because the real setup does not respond instantaneously to angle commands. Neglecting this effect can result in a controller that is too aggressive in simulation or too weak on the experimental system.

The controller input is selected as

$$y_K = \begin{bmatrix} e_x & -\dot{\hat{x}} & e_y & -\dot{\hat{y}} \end{bmatrix}^\top, \tag{45}$$

where

$$e_x = r_x - x, \qquad e_y = r_y - y. \tag{46}$$

This means that the synthesis controller uses the ball position error and the estimated ball velocities. The negative velocity terms correspond to the derivative of the tracking error for a constant reference. Including the velocity information gives the controller additional damping information. For example, the ball may be close to the reference while still moving away from it. In that case, a controller using only position information reacts late, whereas a controller using velocity information can counteract the motion before a large position error develops.

The synthesis control problem is formulated using a weighted generalized plant. The inputs are the shaped reference and disturbance signals,

$$w = \begin{bmatrix} w_{r,x} & w_{r,y} & w_{d,x} & w_{d,y} \end{bmatrix}^\top, \tag{47}$$

and the control input is

$$u = \begin{bmatrix} \alpha & \beta \end{bmatrix}^\top. \tag{48}$$

The performance output is composed of weighted tracking errors, weighted velocity errors, weighted plant outputs, and weighted control inputs:

$$z_p = \begin{bmatrix} W_s e & W_v \dot{e} & W_t y & W_u u \end{bmatrix}. \tag{49}$$

Here, $W_s$ penalizes low frequency tracking error and is therefore mainly used to improve reference tracking and disturbance rejection. The weight $W_v$ penalizes velocity and adds damping to reduce overshoot. The weight $W_t$ penalizes high frequency output response and limits noise amplification. Finally, $W_u$ penalizes excessive plate angle commands.

The design requirements used for the weighting filters were selected based on the experimental behaviour of the setup. The expected reference amplitude is chosen as

$$r_{\max} = 0.20 \text{ m}. \tag{50}$$

The expected velocity scale is chosen as

$$v_{\max} = 0.15 \text{ m/s}. \tag{51}$$

The maximum plate angle used in the weighting design is selected as

$$\alpha_{\max} = \beta_{\max} = 3.0^\circ. \tag{52}$$

The reference bandwidth is selected as $0.07$ Hz, the desired sensitivity bandwidth as $0.095$ Hz, the input weight crossover as $0.05$ Hz, and the complementary sensitivity roll off frequency as $0.7$ Hz. The maximum sensitivity peak is set to

$$M_s = 1.65. \tag{53}$$

The low frequency sensitivity parameter is selected as

$$\varepsilon_S = 0.085. \tag{54}$$

A smaller value of $\varepsilon_S$ increases the penalty on low frequency position error. This improves disturbance rejection, but also makes the controller more aggressive. Therefore, this value is tuned together with the velocity and input weights to avoid excessive overshoot.

This structure allows slow plate angle commands for tracking and disturbance rejection, while penalizing high frequency control action. This is important because high frequency angle commands can excite unmodeled actuator dynamics and amplify measurement noise.

The $H_\infty$ synthesis problem is then written as

$$\min_K \left| F_l(P, K) \right|_\infty, \tag{55}$$

where $P$ is the weighted generalized plant and $F_l(P, K)$ denotes the lower linear fractional transformation between the plant and the controller. The controller is synthesized using

$$K = \texttt{hinfsyn}(P, n_{\text{meas}}, n_{\text{cont}}), \tag{56}$$

with

$$n_{\text{meas}} = 4, \qquad n_{\text{cont}} = 2. \tag{57}$$

Thus, the resulting controller has four inputs and two outputs:

$$K : \begin{bmatrix} e_x & -\dot{\hat{x}} & e_y & -\dot{\hat{y}} \end{bmatrix} \mapsto \begin{bmatrix} \alpha & \beta \end{bmatrix}. \tag{58}$$

For implementation, the continuous time synthesis controller is discretized using the outer loop sampling time. The discrete controller is represented as

$$x_K(k+1) = A_K x_K(k) + B_K y_K(k), \tag{59}$$

$$u_{\text{raw}}(k) = C_K x_K(k) + D_K y_K(k). \tag{60}$$

The matrices $A_K$, $B_K$, $C_K$, and $D_K$ are extracted from the synthesized controller and loaded into the Simulink model before execution. The raw controller output is saturated before being sent further in the control loop. In the implementation, the saturation limits are chosen as

$$-6^\circ \leq \alpha \leq 6^\circ, \qquad -6^\circ \leq \beta \leq 6^\circ. \tag{61}$$

The saturated controller output is therefore

$$u_{\text{sat}}(k) = \text{sat}\left(u_{\text{raw}}(k)\right). \tag{62}$$

Because the controller is dynamic, saturation can lead to windup of the internal controller states. To reduce this effect, anti-windup is added to the controller implementation. The anti-windup error is defined as

$$e_{\text{aw}}(k) = u_{\text{sat}}(k) - u_{\text{raw}}(k). \tag{63}$$

This error is fed back into the controller state update as

$$x_K(k+1) = A_K x_K(k) + B_K y_K(k) + K_{\text{aw}} B_{\text{aw}} e_{\text{aw}}(k). \tag{64}$$

The matrix $B_{\text{aw}}$ is computed as a regularized pseudo-inverse of the controller output matrix,

$$B_{\text{aw}} = C_K^\top \left(C_K C_K^\top + \epsilon_{\text{aw}} I\right)^{-1}. \tag{65}$$

The scalar $K_{\text{aw}}$ determines the strength of the anti-windup correction. A large value makes the controller state recover faster from saturation, but it can also introduce aggressive corrections. Therefore, the anti-windup gain is kept small. A one-sample delay is included in the anti-windup path to avoid an algebraic loop in [8, App.].

After saturation, a rate limiter is applied to the angle command. This limits how fast the commanded plate angles are allowed to change. The rate limiter is not intended to change the steady-state controller behaviour, but it prevents sudden angle commands from exciting the actuator dynamics. The rate limit used in the experiments is selected as

$$\dot{u}_{\max} = 12^\circ/\text{s}. \tag{66}$$

The resulting discrete controller is implemented in a MATLAB Function block, where the controller state is stored and updated at each sample instant. The controller output is then passed to the `AngleToPosition` block. This converts the commanded angles $\alpha$ and $\beta$ into three actuator position references. These references are applied to the inner actuator loops.

### *D. Linear Quadratic Regulator*

Linear Quadratic Regulator (LQR) is an optimal state-feedback control method for linear systems. It computes a control gain $K$ that minimizes a quadratic cost function. A general discrete state space is given by

$$x_{k+1} = Ax_k + Bu_k$$

for which an optimal control law $u_k = -Kx_k$, for stabilization of the origin, and $u_k = -K(x_k - r_k)$, for reference tracking, can be found by minimizing the quadratic cost in Eq. (67).

$$J = \sum_{k=0}^{\infty} (x_k^\top Q x_k + u_k^\top R u_k) \tag{67}$$

The objective is to minimize the infinite horizon cost, with $Q \succeq 0$ and $R \succ 0$. The optimal control gain is computed using Eq. (68).

$$K = (B^\top P B + R)^{-1} B^\top P A \tag{68}$$

The $P$ matrix is the solution to the Discrete Algebraic Riccati Equation (DARE), which is presented in Eq. (69).

$$P = A^\top P A + Q - A^\top P B (B^\top P B + R)^{-1} B^\top P A \tag{69}$$

A solution for optimal gain $K$ can be found using the MATLAB command in Eq. (70).

$$K = \texttt{dlqr}(A, B, Q, R) \tag{70}$$

The matrices $Q$ and $R$ are used as tuning parameters. The matrix $Q$ determines how strongly deviations in each state are penalized. Increasing $Q$ leads to faster state-regulation.

The matrix $R$ can be used to penalize the control effort. Meaning, a larger $R$ results in less aggressive control inputs. Therefore, leading to smoother but slower control.

With this understanding, the LQR control method can now be applied to the control of the ball. The ball position needs to be regulated, whereas the velocity of the ball is less important. However, a weight on the velocity states is desirable to ensure smoother motion.
The $R$ matrix can be used to ensure that the required angle is not too large. Overall, the tuning for LQR control of the ball can be summarized, as in Table VII.

The final tuned LQR weighting matrices are given by:

TABLE VII: Effect of LQR weighting matrices

| Parameter | Effect Summary |
|---|---|
| $Q$ | ↑: faster response, higher control effort<br>↓: slower response, lower control effort |
| $R$ | ↑: smoother inputs, slower response<br>↓: more aggressive control, faster response |

$$Q = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 100 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix} \quad R = \begin{bmatrix} 700 & 0 \\ 0 & 700 \end{bmatrix} \tag{71}$$

### *E. Model Predictive Control*

Model predictive control (MPC) is an optimization-based control method that computes the optimal control input online at each time step. This optimization allows for constraints on its states and control inputs. In order to apply MPC to the ball and plate system, these constraints need to be defined. A constraint is placed on the ball such that it cannot leave a square of 30x30 cm. This makes sure the ball cannot reach the outer wall.

$$\mathbb{Y} := \left\{ y \in \mathbb{R}^2 : \begin{bmatrix} -0.15 \\ -0.15 \end{bmatrix} \leq y \leq \begin{bmatrix} 0.15 \\ 0.15 \end{bmatrix} \right\} \tag{72}$$

Furthermore, the plate cannot be commanded to any angle; the maximum angle in $\alpha$ and $\beta$ is set to $10°$.

$$\mathbb{U} := \left\{ u \in \mathbb{R}^2 : \begin{bmatrix} -10° \\ -10° \end{bmatrix} \leq u \leq \begin{bmatrix} 10° \\ 10° \end{bmatrix} \right\} \tag{73}$$

In order to make the optimization more efficient, extra constraints are placed on the velocity of the ball. This way, all states of the system have finite constraints. Because the velocity of the ball does not have to be constrained tightly, this is set high enough to be sure it does not truly affect the optimization result. This leads to the following state constraints.

$$\mathbb{X} := \left\{ x \in \mathbb{R}^4 : \begin{bmatrix} -0.15 \\ -1 \\ -0.15 \\ -1 \end{bmatrix} \leq x \leq \begin{bmatrix} 0.15 \\ 1 \\ 0.15 \\ 1 \end{bmatrix} \right\} \tag{74}$$

In order to design the MPC controller, the Multi-Parametric Toolbox (MPT3) is applied [2]. This allows the use of polyhedrons to define constraints. To use this toolbox, the constraints must be rewritten to

$$\mathbb{Y} := \left\{ y \in \mathbb{R}^2 | H_y y \leq h_y \right\}$$
$$H_y = \begin{bmatrix} -1 & 0 \\ 1 & 0 \\ 0 & -1 \\ 0 & 1 \end{bmatrix} \quad h_y = \begin{bmatrix} 0.15 \\ 0.15 \\ 0.15 \\ 0.15 \end{bmatrix} \tag{75}$$

$$\mathbb{U} := \left\{ u \in \mathbb{R}^2 | H_u u \leq h_u \right\}$$
$$H_y = \begin{bmatrix} -1 & 0 \\ 1 & 0 \\ 0 & -1 \\ 0 & 1 \end{bmatrix} \quad h_y = \begin{bmatrix} 10° \\ 10° \\ 10° \\ 10° \end{bmatrix} \tag{76}$$

$$\mathbb{X} := \left\{ x \in \mathbb{R}^4 | H_x x \leq h_x \right\}$$
$$H_x = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad h_y = \begin{bmatrix} 0.15 \\ 0.15 \\ 1 \\ 1 \\ 0.15 \\ 0.15 \\ 1 \\ 1 \end{bmatrix}. \tag{77}$$

In order to ensure convergence of the MPC controller, the terminal set of states must be defined. That is, the invariant set of states in which all constraints are met. In order to determine this set, the terminal control law must be computed first. This is the optimal control law without taking constraints into account. This is computed using Matlabs `dlqr()` function, which returns the terminal cost $P$ and optimal control gain $K$. Using this, the control admissible set can be defined as

$$\mathbb{X}_{\text{UA}} := \left\{ x \in \mathbb{R}^4 | (H_u K) x \leq h_u \right\}. \tag{78}$$

Then, the total constraint admissible set can be defined as the intersection of this control admissible set and the set of states.

$$\mathbb{X}_{\text{CA}} = \mathbb{X} \cap \mathbb{X}_{\text{UA}} \tag{79}$$

To ensure that all states in the terminal set will stay inside the terminal set, the invariant set of the constraint admissible set is computed.

$$\mathbb{X}_T := \{ x \in \mathbb{X}_{\text{CA}} | (A - BK) x \in \mathbb{X}_{\text{CA}} \} \tag{80}$$

These steps have been implemented in the MPT3 toolbox and result in the following polyhedron.

$$\mathbb{X}_T := \left\{ x \in \mathbb{R}^4 | H_T x \leq h_T \right\} \tag{81}$$

The set is plotted in Fig. 10. This shows the position-velocity space in both the $x$ and $y$ directions. From this, it is clear that the whole plate is inside the terminal set as long as the

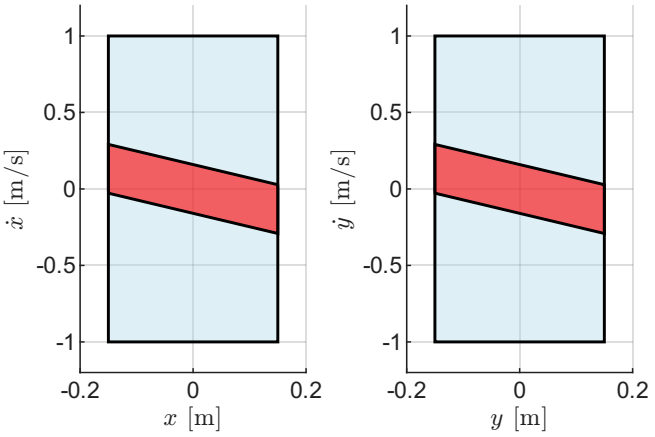


Fig. 10: Terminal set $\mathbb{X}_T$ in x direction (left) and y direction (right).

ball's velocity is small enough. Using these constraints, the condensed MPC problem is given as

$$\begin{aligned}\min_{\Delta U_k} J = \min_{\Delta U_k} &(X_k - R_k)^T \Omega (X_k - R_K) + U_k^T \Psi U_k \\ &+ \Delta U_K^T \Psi_\Delta \Delta U_K \\ &\text{s.t.} \\ &X_k = \Phi \hat{x}_{k|k} + \Gamma U_k \\ &Y_k = \Theta X_k \\ &U_k = M_1 u(k-1) + M_2 \Delta U_k \\ &Y_k \in \mathbb{Y} \\ &U_k \in \mathbb{U} \\ &X_N \in \mathbb{X}_T.\end{aligned} \tag{82}$$

With,

$$\Phi = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \Gamma = \begin{bmatrix} B & 0 & \dots & 0 \\ AB & B & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \dots & B \end{bmatrix} \tag{83}$$

$$\Theta = \begin{bmatrix} C & 0 & \dots & 0 \\ 0 & C & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & C \end{bmatrix} \tag{84}$$

$$\Omega = \begin{bmatrix} Q & & & \\ & Q & & \\ & & \ddots & \\ & & & P \end{bmatrix}, \quad \Psi = \begin{bmatrix} R & & & \\ & R & & \\ & & \ddots & \\ & & & R \end{bmatrix}$$

$$\Psi_\Delta = \begin{bmatrix} R_\Delta & & & \\ & R_\Delta & & \\ & & \ddots & \\ & & & R_\Delta \end{bmatrix} \tag{85}$$

$$M_1 = \begin{bmatrix} I_{n_u} \\ I_{n_u} \\ \vdots \\ I_{n_u} \end{bmatrix} \; M_2 = \begin{bmatrix} I_{n_u} & 0_{n_u \times n_u} & 0_{n_u \times n_u} & \dots & 0_{n_u \times n_u} \\ I_{n_u} & I_{n_u} & 0_{n_u \times n_u} & \dots & 0_{n_u \times n_u} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ I_{n_u} & I_{n_u} & I_{n_u} & \dots & I_{n_u} \end{bmatrix} \tag{86}$$

and,

$$\begin{aligned} X_k &= \{x_{1|k}, x_{1|k}, \cdots, x_{N|k}\} \\ R_k &= \{r_{1|k}, r_{2|k}, \cdots, r_{N|k}\} \\ Y_k &= \{y_{1|k}, y_{1|k}, \cdots, y_{N|k}\} \\ U_k &= \{x_{0|k}, x_{1|k}, \cdots, x_{N-1|k}\} \end{aligned} \tag{87}$$

The weighting matrix $\Omega$ has the terminal cost $P$ in the final diagonal position to ensure stability. In this optimization the decision variable is $\Delta U_k$, which is converted to absolute control effort $U_k$ and states $X_k$ in the equality constraints, which allows for the inequality constraints. Furthermore, the difference between the state and the reference is penalised over the total prediction horizon, which allows for look-ahead reference tracking.

This controller is implemented in Matlab using the MPC block in Simulink [7]. This takes the discrete state space model, the constraints and the tuning parameters $Q$, $R$, $N$ and $N_c$. Where $N_c$ is the control horizon. This block uses the `quadprog` algorithm to solve the optimization defined above, and takes the first optimal control input. This is done for each time step $k$. Using the block allows a simple and effective real-time implementation that can be compiled to the dSPACE control system. However, this implementation does have some drawbacks. In order to simplify the controller for real-time implementation, the terminal cost $P$ and terminal set $\mathbb{X}_T$ are not included. To still guarantee feasibility, the block automatically applies soft constraints. These allow the constraint to be exceeded slightly if that is the only feasible solution.

The controller is tested on the simulation described below and tuned to the following weights.

$$Q = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 100 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix} \quad R = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix} \tag{88}$$

$$R_\Delta = \begin{bmatrix} 200 & 0 \\ 0 & 200 \end{bmatrix} \tag{89}$$

Here, an extra weight is put on the position states as the measurement for these is more accurate and less noisy. Moreover, it is more important to minimize error in the position rather than the velocity for accurate reference tracking.

## VI. Implementation

In this section, the rate difference between the outer-loop and inner-loop controllers is addressed. Furthermore, the implementation of the 3D camera is discussed.

### A. Rate transition

The inner control loop runs at a sample frequency $f_s = 1000$ Hz, whereas the camera runs much slower at $f_s = 50$ Hz. This difference in sample frequencies has to be implemented carefully. If the angle commands from the outer loop were directly applied as stepwise actuator references, the inner loop would receive discontinuous position setpoints. This can excite the actuator dynamics and lead to unnecessary overshoot, vibrations, or aggressive motor currents.

To avoid this, a first-order low-pass filter with a cutoff frequency of 50 Hz is used between the outer and inner control loops. The low-pass filter ensures that the steps commanded by the outer loop controller are smoothed.

This approach allows the outer loop to operate at a lower frequency, which is suitable for the camera-based ball position measurements, while the inner actuator loop can still run at a high frequency to accurately track the actuator position references. The low-pass filter is therefore important for combining the slow outer loop with the fast actuator control loop without introducing discontinuities or timing inconsistencies.

### B. 3D Camera

The 3D camera is responsible for capturing the 3D position of the ball on the plate with respect to the operating point, which is the midpoint of the linear actuator's stroke range. The camera used in this case is an Intel RealSense D435, which is implemented using Python.

This subsection will go over two main components of the 3D camera:

1) The ball detection and tracking algorithm in the 2D space $X$-$Y$
2) Depth measurement of the ball to get its position in the $Z$-axis

*1) Ball detection and tracking in the 2D space $X$-$Y$:* 2D detection of the ball is pretty straightforward with a combination of image processing and filtering. The ball is detected based on brightness: when an image is acquired using the camera, the first step is to convert the RGB into grayscale using cv2, which is a Python interface for the OpenCV library [5]. This function converts each colored pixel into a shade of gray $0 \leq I \leq 255$ where $0 =$ black and $255 =$ white.

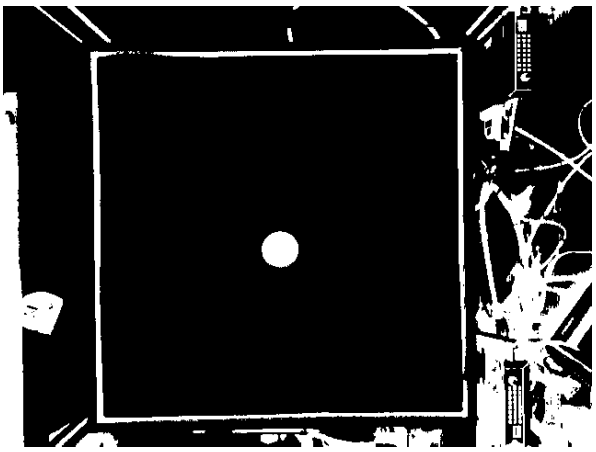

Fig. 11: Thresholded grayscale image

Using the cleaned binary image, the algorithm applies contour detection, where a contour is simply the boundary of a connected object as depicted in Fig. 12.

From the multiple contours detected, filtering is then applied to eliminate false positives by using area filtering and then circularity filtering to detect which contour resembles the ball the most.

Once a valid candidate is found, its centroid is computed to retrieve the ball's position in $x$ and $y$ coordinates.

*2) Depth measurement:* Depth measuring is a bit more complex than 2D detection of the ball. It is important to first quickly go over the working principle of the 3D camera to understand how it captures a depth profile.

The Intel RealSense D435 measures depth using active infrared stereo vision. Two infrared cameras observe the scene from slightly different viewpoints while an infrared projector projects a textured dot pattern onto the environment. The onboard vision processor identifies corresponding features in

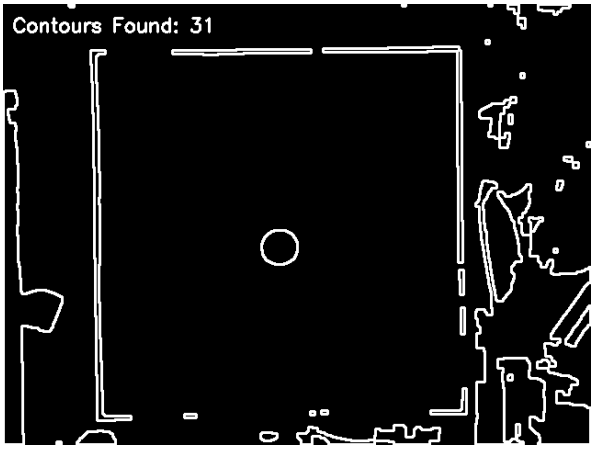


Fig. 12: Contour detection


(a) Area filtering (b) Circularity filtering

Fig. 13: Image processing filters

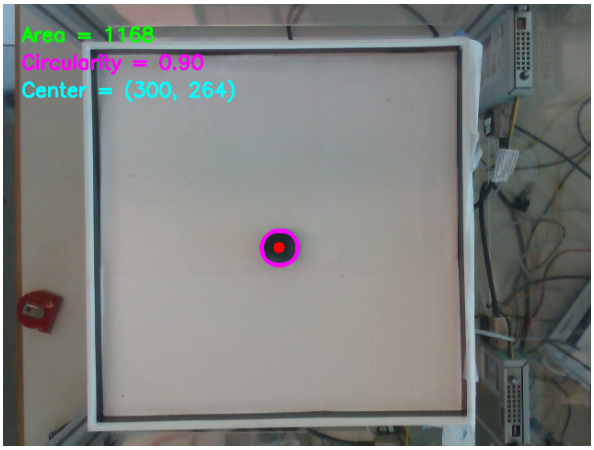


Fig. 14: Final ball detection in 2D space

both infrared images and computes their disparity. Using the known baseline $B$ between the cameras and the camera focal length $f$, the disparity $d$ is converted into depth $Z$ through stereo triangulation, following this relationship:

$$Z = \frac{fB}{d} \tag{90}$$

The resulting depth map provides the distance from the camera to every visible point in the scene, which can then be accessed directly through the RealSense SDK [6].

Currently, the depth measurement is entirely relying on the 2D tracking of the ball. When the ball is found, and the coordinates of the center are calculated, for the depth,

a region of interest (ROI) in the form of a small circle is defined around that center. And there, the depth is averaged out in this ROI to smooth the $Z$ measurement.
In addition to that, the Intel RealSense temporal filter is applied to each acquired depth frame before each extraction. The filter reduces frame-to-frame measurement noise by combining the current depth image with the information from previous frames. This temporal averaging suppresses random fluctuations while preserving slowly varying depth values.

Two parameters control the filter behaviour: the smoothing coefficient $\alpha$, which determines the amount of temporal averaging, and the depth threshold $\delta$, which allows large depth changes to be treated as genuine motion rather than noise. In this work, $\alpha = 0.6$ and $\delta = 20$ were selected as a compromise between measurement stability and responsiveness.

## VII. Simulation

In order to validate both the inner and outer loop controllers defined above, a Simulink simulation is built. This model is designed to mirror the experiment model that is used below for the implementation on the true system. To simulate the inner loop, the kinematics, actuator models, and inner loop controllers defined above are used. This way, the outer loop controllers can be tested and tuned.

### A. Visualization

In order to visually verify the systems performance, a `PlateVisualizer` object is implemented in the Simulink model. This visualization includes buttons to start and stop the inner and outer loops and a reset for the ball position.

### B. Square reference

In order to analyze the performance of all 4 controllers, they are simulated following a square reference. This reference first moves the ball from the center of the plate to the top right corner and then follows a quintic polynomial square reference with a width of 20cm.

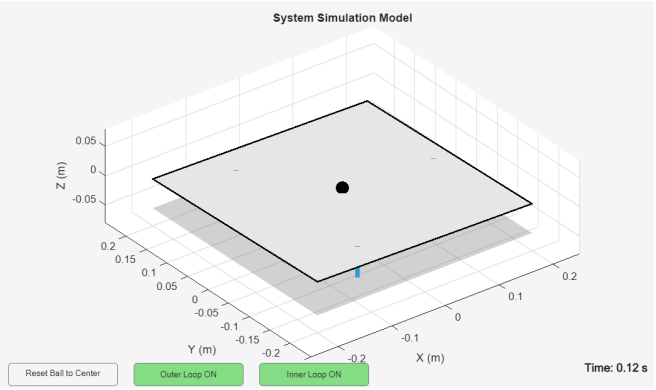


Fig. 15: Plate visualization of Simulink model

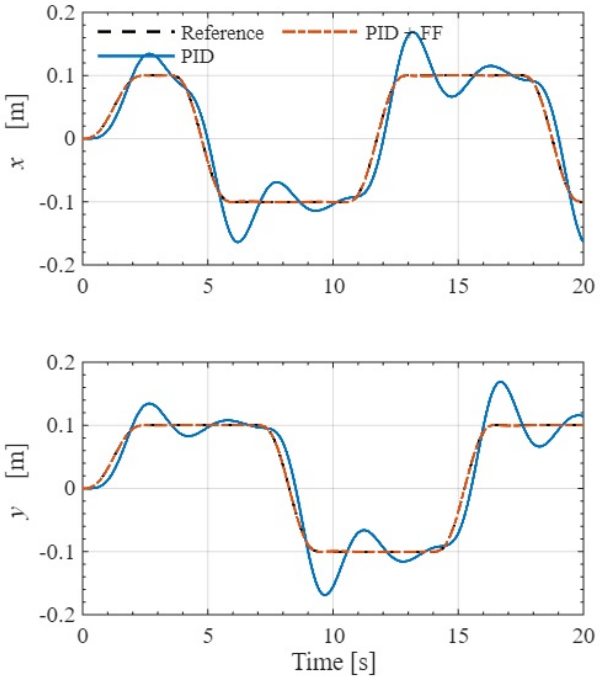


Fig. 16: Simulated PID reference tracking with and without feedforward control.

### C. Performance metrics

To validate and compare each controller, three different performance metrics are computed for each trial. Firstly, the root mean square (RMS) errors evaluated, which is the RMS value of the Euclidean distance between the ball position and the reference. This metric gives an indication of the average performance of the controller.

$$e_{RMS} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}((x_k - x_{ref,k})^2 + (y_k - y_{ref,k})^2)} \quad (91)$$

Secondly, the peak value of the Euclidean distance between the ball position and the reference is evaluated. This indicates the worst-case performance of the controller.

$$e_{max} = \max_{k\in\{1,\dots,N\}} \sqrt{(x_k - x_{ref,k})^2 + (y_k - y_{ref,k})^2} \quad (92)$$

Lastly, the total control energy is evaluated in order to compare the control effort needed to control the ball.

$$E_u = T_s \sum_{k=1}^{N} (\alpha_k^2 + \beta_k^2) \quad (93)$$

### D. Feedforward performance

In order to test the improvement provided by the feedforward control, the PID controller simulated both with and without feedforward control. Fig 16 shows the reference tracking performance. From this, it is clear that the PID controller can follow the reference roughly, but adding feedforward improves this performance significantly. Table VIII

shows the performance metrics for both controllers. From this, it is clear that the feedforward improved performance is all three metrics.

TABLE VIII: Performance comparison with and without feedforward control.

| **Controller** | $e_{RMS}$ [m] | $e_{max}$ [m] | $E_u$ [rad$^2$·s] |
|---|---|---|---|
| PID | 0.03754 | 0.06983 | 0.0135 |
| PID + FF | 0.00035 | 0.00063 | 0.0051 |

### *E. Reference tracking results*

The performance of each controller is shown in fig. 17. From this, it is clear that all four controllers can follow the reference. The controller that use feedforward (PID and LQR) perform very well. The MPC controller has some oscilation around the reference. The $H_\infty$ controlller has some small overshoot but settles quickly.

TABLE IX: Simulation performance comparison for all 4 controllers.

| **Controller** | $e_{RMS}$ [m] | $e_{max}$ [m] | $E_u$ [rad$^2$·s] |
|---|---|---|---|
| PID + FF | 0.00035 | 0.00063 | 0.0051 |
| Hinf | 0.01396 | 0.03379 | 0.0048 |
| LQR + FF | 0.00774 | 0.01972 | 0.0060 |
| MPC | 0.04445 | 0.08331 | 0.0064 |

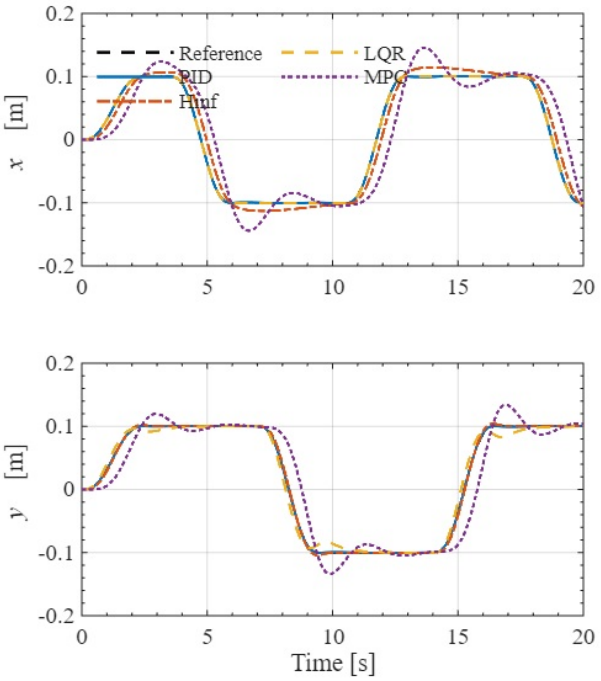


Fig. 17: Simulation result for all 4 controllers following a square reference of width $R = 0.1$.

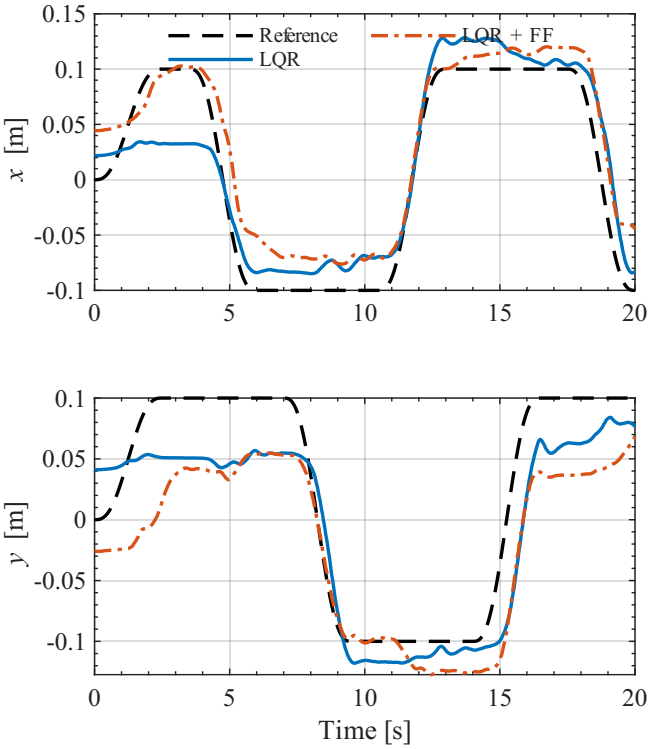


Fig. 18: Experimental results, comparing LQR control with and without feedforward.

## VIII. Experimental results

Now that it is shown that all four controllers are stable and perform well in simulation, they can be applied to the experimental system. The system is controlled using a Simulink model that has the same structure as the simulation model. This model is automatically compiled to C code and built on a dSPACE controller. The system can be interacted with using the control desk environment, which is used to trigger things like moving the plate to operating position, enabling controllers, and recording data.

### *A. Feedforward performance*

In order to test the feedforward controller on the real setup, the LQR controller is applied both with and without feedforward enabled. From Fig. 18 shows the results of these tests. From this, it is clear that the LQR controller can roughly track the reference. However, enabling feedforward does not improve the performance greatly. From Table X, it can be seen that both the error performance and control effort increase slightly.

As can be expected, this result is very different than the excellent performance achieved in simulation. This is because in simulation, the acceleration feedforward is a perfect inverse of the plant, which means it perfectly predicts the acceleration needed to follow the reference. In the true setup, the plant is not as perfect as the model. The setup has a plate that is not perfectly flat and level. The true plate might be slightly bent, and its top layer is not flat; it has some significant bumps. Due to these mismatches between

the model and the true system, the acceleration computed in the feedforward will not always be optimal.

TABLE X: Performance comparison with and without feed-forward control on experimental setup.

| **Controller** | $e_{RMS}$ [m] | $e_{max}$ [m] | $E_u$ [rad$^2$·s] |
|---|---|---|---|
| LQR | 0.04550 | 0.09710 | 125.0407 |
| LQR + FF | 0.05055 | 0.10274 | 129.8080 |

### *B. Reference tracking results*

Fig. 19 shows the reference tracking performance for all 4 of the outer loop controllers. Based on this, it can be concluded that all 4 controllers are stable and tracking the reference well. Table XI shows the performance metrics for all 4 controllers.

TABLE XI: Experimental performance comparison for all 4 controllers.

| **Controller** | $e_{RMS}$ [m] | $e_{max}$ [m] | $E_u$ [rad$^2$·s] |
|---|---|---|---|
| PID + FF | 0.03648 | 0.08203 | 221.5650 |
| $H_\infty$ | 0.05026 | 0.10370 | 134.6011 |
| LQR + FF | 0.05365 | 0.10972 | 129.8685 |
| MPC | 0.04199 | 0.10948 | 123.3328 |

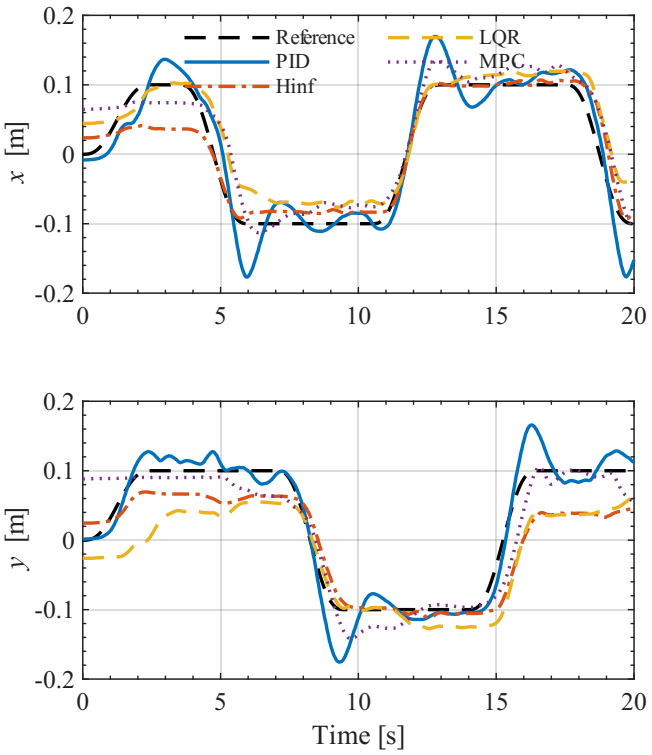


Fig. 19: Experimental results, comparing all 4 controllers.

The PID controller has very good error performance but does oscillate around the reference in the in the steady state areas. The control effort of this controller is relatively high, which implies that the plate is being moved unnecessarily during tracking.

The $H_\infty$ controller has slightly higher RMS error, which is mostly explained by a relatively large steady state error. It also has relatively high control energy, which is due to some small oscillations during tracking.

The LQR controller has similar error performance to the $H_\infty$ controller but has requires slightly lower control energy to achieve this.

The MPC controller has lower RMS error performance than the LQR controller and even lower control energy required.

The considerable increase in control effort of the PID controller can be explained by the fact that this controller is the only one that does not have the possibility of tuning the control effort itself. By tuning the $R$ matrices in the LQR and MPC designs and the $W_u$ weight in the $H_\infty$ design, control effort is minimized.

It is important to note that, although the controllers work as expected, plate vibrations were present during operation of the outer loop controllers. These vibrations will be analyzed in Section IX.

### *C. reference tracking in z direction*

A challenge encountered during this project is the control of the z-direction. With the implementation of a new 3D camera as a sensor to the system, this allows for the opportunity to measure depth as explained above in VI
The idea is to prove that the z-direction can be controlled as an independent direction. It is tested by applying a simple sine reference to the z-direction using a P controller with $P = 1$, which yields the following result in Fig. 20. The output appears decent, especially considering that only a P controller is applied.

Therefore, this proves that the Z-direction can indeed be controlled, and there is much room for improvement in the controller. This validates the idea that 3D reference tracking can be applied to this system quite effectively.

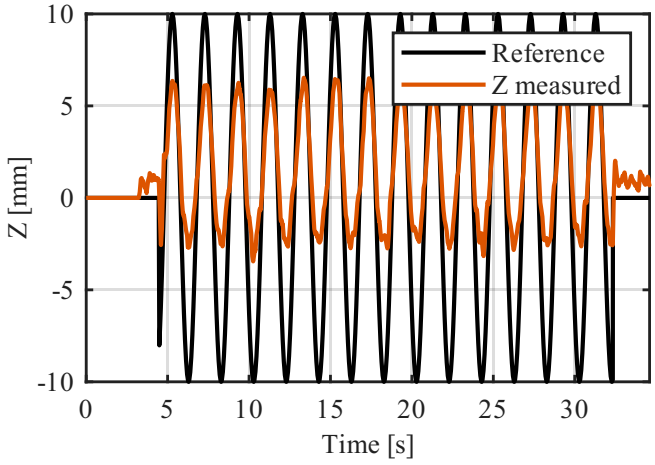


Fig. 20: Output position in $Z$ compared to the sine reference applied

## IX. Discussion

In this section, the plate vibrations are analyzed, and improvements are suggested. Furthermore, the performance of the outer-loop controllers is addressed.

### A. Plate vibrations during outer-loop control

All outer-loop controllers tested on the experimental setup showed vibrations of the plate.
Subsequently, the vibrations were analyzed to find the root cause. The Power Spectral Density (PSD) of both the error signals and input signals to the motors is provided in Fig. 21.

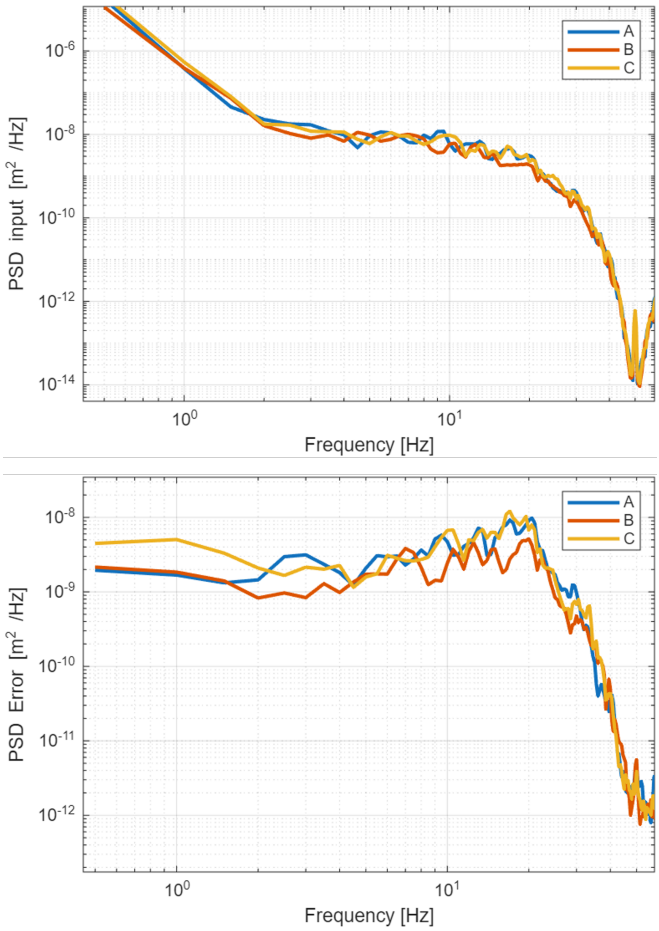


Fig. 21: The PSD of the motor inputs and error signals.

The PSD shows that the error signals have more energy around 20 Hz, which corresponds to the observed vibration during experiments. Interestingly, the PSD of the input to the inner loop does not contain high energy near 20 Hz. This insight shows that the vibration is induced in the inner loop.
The Sensitivity plot of the inner-loop controllers, presented in Fig. 22, all show a peak above 0 dB near 20 Hz. Thus, the 20 Hz content in the input is amplified, confirming the high energy content in the error PSD. Although the outer-loop controllers themselves do not introduce significant vibrations, the commanded angles had 20 Hz content that is amplified by the inner-loop.

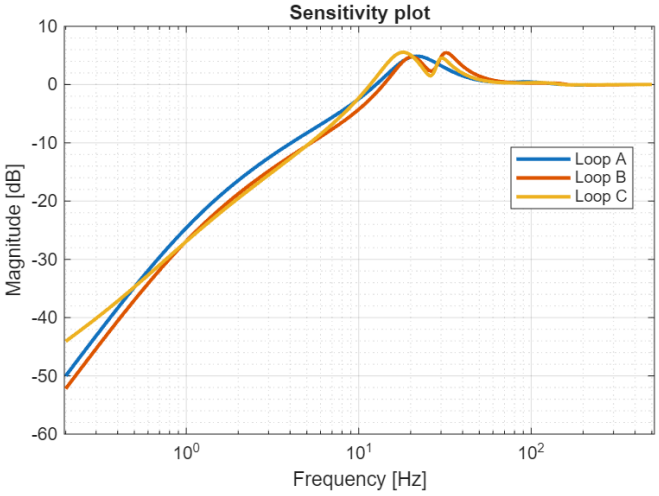


Fig. 22: The Sensitivity plots motors A, B, and C.

The vibrations could be reduced by introducing command shaping between the outer and inner loops. This command shaper can reduce the 20 Hz content present in commanded outer loop angles.
The inner-loop controller could be altered, however, shifting or reducing the peak near 20 Hz is quite challenging due to the waterbed effect.

### B. Performance improvements outer loop controllers

In general, all 4 outer loop controllers perform well. But their performance can be improved further by spending more time on tuning their parameters. Furthermore, the steady state performance of the LQR can be improved by adding integrator action. The MPC controller currently uses both weights on the absolute control effort $U_k$ and the relative control effort $\Delta U_k$. If the absolute weight were removed, the weight on $\Delta U_K$ would act as an integrator; currently, this effect is prevented by the absolute weight. This was not yet applied this way due to time constraints.
The integrator can be added to LQR by using LQI. LQI is an extension of LQR. This extension can be obtained by augmenting the ball dynamics with an integrator state, with the new state $x_a$ given by

$$x_a = \begin{bmatrix} x \\ x_I \end{bmatrix}, \tag{94}$$

where $x_I$ is the integral of the tracking error. The augmented system is given by

$$x_{a,k+1} = \begin{bmatrix} A & 0 \\ -C & I \end{bmatrix} x_{a,k} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k. \tag{95}$$

The LQR design procedure presented in Section V-D can be applied to the new augmented state space. The $Q$ matrix should now be updated to

$$Q = \begin{bmatrix} Q_x & 0 \\ 0 & Q_I \end{bmatrix}, \tag{96}$$

where the matrix $Q_x$ is tuned exactly for the ball-dynamics and $Q_I$ is the new weighting matrix that can penalize the integral state. The $R$ matrix remains exactly the same as for LQR. The resulting control law is given by

$$u_k = -K_x x_k - K_I x_{I,k}, \tag{97}$$

where $K_x$ is the state-feedback gain and $K_I$ is the integral gain.

## X. Conclusions

This report presented the modeling, identification, simulation, and experimental implementation of the ball and plate setup. The system was divided into several subsystems, consisting of actuator dynamics, plate kinematics, ball dynamics, camera measurement, inner loop, and outer loop control. This made it possible to model and validate different parts of the setup separately before integrating them.

Several outer loop control implementations were designed and compared. A PID controller with feedforward was used as a benchmark, while LQR, $H_\infty$ synthesis, and MPC were implemented as more model-based approaches. The simulation results showed that all controllers were able to track the square reference. The feedforward contribution significantly improved the PID tracking performance by compensating for the acceleration required by the reference. In the experimental results, all controllers were also able to stabilize and track the ball position. However, clear differences were observed in the trade-off between tracking accuracy and control effort. The PID controller with feedforward achieved good tracking performance but required a relatively large amount of control energy. For LQR, feed forward was also implemented to further improve tracking performance. The LQR and $H_\infty$ controllers produced stable responses with lower control effort, although their performance was affected by steady state errors and small oscillations. The MPC controller achieved a good compromise between reference tracking and control effort, partly because constraints and input penalties could be included explicitly in the optimization problem.

Lastly, feedback control was implemented on the vertical z-axis. This was shown to be able to use the height measurement from the 3D camera to follow a simple sine wave reference using a proportional gain controller. However, in order to be able to follow a complete 3D reference, more work is needed in reducing camera noise and expanding the control structure.

The experimental implementation also showed that several practical effects are not fully captured by the simplified model. These include camera noise, delays, actuator friction, plate imperfections, and vibrations introduced by the inner loop dynamics. In particular, vibrations around the plate were observed during outer loop operation. The frequency domain analysis indicated that these vibrations were related to the sensitivity behavior of the inner actuator loops. This shows that the performance of the outer loop controller is strongly influenced by the accuracy and robustness of the inner loop control system.

The results also show that experimental implementation requires more than simply applying model-based controller designs. Practical limitations, such as actuator dynamics, measurement noise, sampling rate differences, saturation, and mechanical imperfections, must be considered during implementation and tuning.

## Appendix

### A. Kinematic functions

The function below is used to convert a desired angle to a the actuator set point of the three actuators.

```
function [posA, posB, posC] = angleToPos(alpha,
    beta, psi)
    % Returns the actuator positions required to
        reach the desired angle
    % note: the position is absolute in the global
        frame

    % Physical Constants
    r = 0.17;           % Joint radius (m)
    h = [0; 0; 0.32];  % Resting height vector (m)

    % Joint positions on base and plate (
        equilateral triangle)
    % These match the definitions in your
        posToAngle function
    theta = [0, deg2rad(240), deg2rad(120)];
    B = [r*cos(theta); r*sin(theta); zeros(1,3)]; %
         Base joints (b_i)
    P = [r*cos(theta); r*sin(theta); zeros(1,3)]; %
         Plate joints (p_i)

    % Rotation Matrices (Yaw-Pitch-Roll order)
    Rz = [cos(psi) -sin(psi) 0; sin(psi) cos(psi)
        0; 0 0 1];
    Ry = [cos(beta) 0 sin(beta); 0 1 0; -sin(beta)
        0 cos(beta)];
    Rx = [1 0 0; 0 cos(alpha) -sin(alpha); 0 sin(
        alpha) cos(alpha)];
    R_BtoP = Rz * Ry * Rx;

    % Compute absolute position for each motor
    % Motor 1
    l1_vec = h + (R_BtoP * P(:,1)) - B(:,1);
    posA = norm(l1_vec);

    % Motor 2
    l2_vec = h + (R_BtoP * P(:,2)) - B(:,2);
    posB = norm(l2_vec);

    % Motor 3
    l3_vec = h + (R_BtoP * P(:,3)) - B(:,3);
    posC = norm(l3_vec);
end
```

The function below is used to convert the actuator positions of the three actuators to the angle of the plate.

```
function [alpha, beta, psi] = posToAngle(posA, posB
    , posC)
    % Returns the angles resulting from the given
        position
    % note: the position should be relative from
        the actuators midpoint in the global frame

    % Physical Constants
    r = 0.17;                % Radius to joints (m
        )
    h = 0.322;               % Resting height (m)
    theta = deg2rad(120);    % angle between bases
         (rad)
```

```
    % Reconstruction of the Plate Unit Vectors
    % We find where the joints are in the world
        frame by using the inputs
    % If pos = 0.322, the joints are exactly
        T_height above the base.
    P1_global = [r; 0; h + posA];
    P2_global = [r*cos(theta); r*sin(theta); h +
        posB];
    P3_global = [r*cos(theta); -r*sin(theta); h +
        posC];

    % Find the center of the tilted plate
    % The average of the three joint positions
        gives the center of the motion plate
    center = (P1_global + P2_global + P3_global) /
        3;

    % Vectors from the plate center to the joints
    % This isolates the tilt of the plate surface
    v1 = P1_global - center;
    v2 = P2_global - center;
    v3 = P3_global - center;

    % Compute Unit Vectors
    uX = v1 / norm(v1);                % Plate X-axis
    u_temp = cross(uX, v3);            % Temporary
        vector to find normal
    uZ = u_temp / norm(u_temp);        % Plate Z-axis
        (Normal)
    uY = cross(uZ, uX);                % Plate Y-axis

    % Extract Tait-Bryan Angles
    beta = asin(-uX(3));
    alpha = asin(uY(3) / sqrt(1 - uX(3)^2));
    psi = asin(uX(2) / sqrt(1 - uX(3)^2));
end
```

## B. Models and FRFs for motor B and C

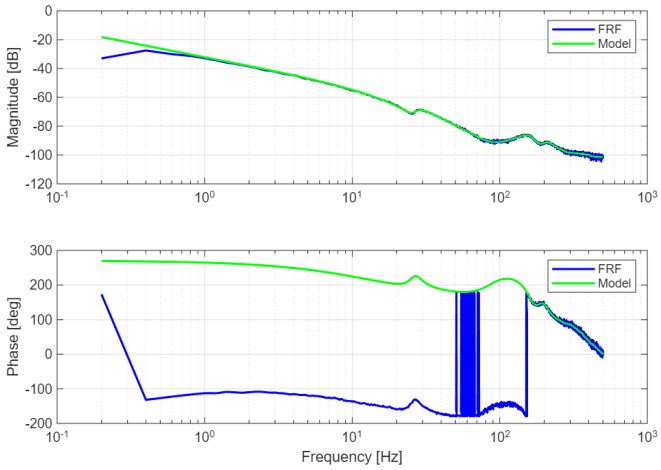


Fig. 23: Bode plot of motor B of the identified model and FRF.

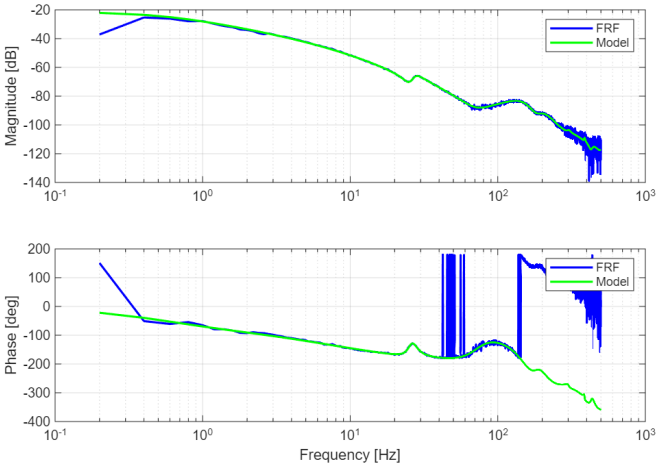


Fig. 24: Bode plot of motor C of the identified model and FRF.

## C. Kalman implementation

```
function x_est = intermittent_kalman(y_meas, u,
    valid_frame, Qkf, Rkf, Ts)

    g = 9.81;
    Ad = [1, Ts, 0,0;
        0, 1, 0, 0;
        0, 0, 1, Ts;
        0, 0, 0, 1];

    Bd = [5/14 * g * Ts^2, 0;
        5/7 * g * Ts, 0;
        0, 5/14 * g * Ts^2;
        0, 5/7 * g * Ts];

    Cd = [1 0 0 0;
        0 0 1 0];


    % Persistent variables keep their state between
        sample steps
    persistent x_hat P

    % Initialization on the very first time step
    if isempty(x_hat)
        x_hat = zeros(4,1);
        P = eye(4) * 0.1;
    end

    % 1. Prediction Step (Always executes)
    x_pred = Ad * x_hat + Bd * u;
    P_pred = Ad * P * Ad' + Qkf;

    % 2. Correction Step (Conditional)
    if valid_frame > 0.5
        % Frame is valid: Run normal Kalman
            Correction
        S = Cd * P_pred * Cd' + Rkf;
        K = P_pred * Cd' / S; % Equivalent to
            P_pred * Cd' * inv(S)

        x_hat = x_pred + K * (y_meas - Cd * x_pred)
            ;

        P = (eye(4) - K * Cd) * P_pred;
    else
        % Frame was dropped: Skip correction, trust
            physics prediction
        x_hat = x_pred;
```

```
        P = P_pred;
    end

    % Output the current state estimate
    x_est = x_hat;
end
```

### D. Quintic Polynomial

```
function path = Quintic_Time(reference_end,
    end_time, Ts, measured_position)

persistent coeffs t_elapsed prev_ref_end
    tf_internal
if isempty(coeffs)
    coeffs = zeros(6, 1); % a0, a1, a2, a3, a4, a5
end
if isempty(t_elapsed)
    t_elapsed = 0;
end
if isempty(prev_ref_end)
    prev_ref_end = reference_end;
end
if isempty(tf_internal)
    tf_internal = 0;
end

% Check if a new trajectory is requested
if reference_end ~= prev_ref_end
    % Reset timer
    t_elapsed = 0;
    tf_internal = end_time;

    % Boundary conditions
    q0 = measured_position;
    qf = reference_end;
    v0 = 0; vf = 0;
    ac0 = 0; acf = 0;



    T = tf_internal;
    M = [1, 0, 0,   0,     0,     0;
         0, 1, 0,   0,     0,     0;
         0, 0, 2,   0,     0,     0;
         1, T, T^2, T^3,   T^4,   T^5;
         0, 1, 2*T, 3*T^2, 4*T^3, 5*T^4;
         0, 0, 2,   6*T,   12*T^2, 20*T^3];

    b = [q0; v0; ac0; qf; vf; acf];

    % Calculate coefficients
    coeffs = M \ b;
end

% Calculate current reference based on elapsed time
t = t_elapsed;

if t <= tf_internal
    % Quintic Equation: path = a0 + a1*t + a2*t^2 +
        a3*t^3 + a4*t^4 + a5*t^5
    path = coeffs(1) + coeffs(2)*t + coeffs(3)*t^2
        + coeffs(4)*t^3 + coeffs(5)*t^4 + coeffs(6)
        *t^5;

    % Increment time for the next step
    t_elapsed = t_elapsed + Ts;
else
    % Hold the final position once trajectory is
        finished
    path = reference_end;
end

% Update memory for next iteration
prev_ref_end = reference_end;

end
```

### E. $H_\infty$ Controller Implementation

Listing 1: MATLAB Function implementation of the discrete $H_\infty$ controller with saturation, anti-windup and rate limiting.

```
function [u_cmd, u_raw, u_box, sat_flag, xK_norm] =
     Hinf( ...
e_x, ev_x, e_y, ev_y, ...
reset, ...
Arob, Brob, Crob, Drob, ...
Baw, Kaw, ...
umin, umax)

% Inputs:
%   e_x, ev_x, e_y, ev_y = [r_x-x; -xdot; r_y-y; -
    ydot]
%   reset                = 1 when outer loop
    disabled
%
% Outputs:
%   u_cmd   = final command to plant / AngleToPos
%   u_raw   = unconstrained Hinf output
%   u_box   = amplitude-saturated Hinf output
    before rate limit
%   sat_flag = logical flag indicating saturation
    or rate limiting
%   xK_norm = norm of the controller state

persistent xK u_prev initialized

nK = size(Arob,1);

if isempty(initialized)
xK = zeros(nK,1);
u_prev = zeros(2,1);
initialized = true;
end

u_raw = zeros(2,1);
u_box = zeros(2,1);
u_cmd = zeros(2,1);
sat_flag = false;
xK_norm = 0;

% Reset controller state while outer loop is off
if reset ~= 0
xK = zeros(nK,1);
u_prev = zeros(2,1);
return;
end

%% Inputs to Hinf controller

yK = [e_x; ev_x; e_y; ev_y];

u_min = [umin(1); umin(2)];
u_max = [umax(1); umax(2)];

%% Unconstrained controller output

u_raw = Crob*xK + Drob*yK;

%% Amplitude saturation

u_box = min(max(u_raw, u_min), u_max);

%% Rate limiting

Ts_outer = 0.02;              % Hinf sample time [s
    ]
```

```
maxRate  = deg2rad(20.0);     % maximum angle rate [
    rad/s]

du_max = maxRate * Ts_outer;

du = u_box - u_prev;

du_limited = min(max(du, [-du_max; -du_max]), ...
[ du_max;  du_max]);

u_cmd = u_prev + du_limited;

% Safety saturation after the rate limiter
u_cmd = min(max(u_cmd, u_min), u_max);

%% Anti-windup

du_aw = u_cmd - u_raw;

x_nom = Arob*xK + Brob*yK;
x_next = x_nom + Kaw*(Baw*du_aw);

% Numerical guard
if any(~isfinite(x_next)) || norm(x_next) > 1e8
x_next = zeros(nK,1);
u_cmd = zeros(2,1);
end

xK = x_next;
u_prev = u_cmd;

sat_flag = any(abs(u_cmd - u_raw) > 1e-10);
xK_norm = norm(xK);

end
```

In general, all team members helped each other a lot. The points below represent which part of the project the person was responsible for; this generally does not mean that the person was the only one working on it. The part of the report belonging to the part is generally also written by that person.

### F. Alexis Santucci

- 3D Camera Implementation
- 2D Detection and Tracking of the ball
- Calibration procedure for the camera via a Python code
- Height measurement of the Ball using the 3D camera depth measurement
- PID controller for the outer loop

### G. Costin Stanicel

- Synthesis and implementation of the $H_\infty$ controller
- Practical implementation, different discussions, and theoretical work during experimental sessions
- Helped with the first principle modeling of ball kinematics
- Improvement of Simulink model
- Cleaning of the implementation of Kalman filtering
- Inner loop tuning

### H. Thomas Roose

- Parts of the first principles modeling of actuators and ball.
- kinematic function posToAngle and AngleToPos.
- Simulink simulation that mirrors the given experiment model, including visualization.
- Design and implementation of the MPC outer loop controller.
- Step response model validation.
- Inner loop integrator implementation, including anti-windup saturation and reset.

### I. Simon Rosenberg

- System identification
- Kalman filter
- LQR
- Practical implementations on hardware, such as reference generators
  - Quintic polynomial
  - Square and circular references outer-loop
- Innerloop control on setup
- Feedforward control

## References


[1] Eindhoven University of Technology. *ShapeIt*. https://dsdwiki.wtb.tue.nl/wiki/Home_of_ShapeIt. Accessed: 2026-06-24. 2026.

[2] M. Herceg et al. "Multi-Parametric Toolbox 3.0". In: *Proc. of the European Control Conference*. http://control.ee.ethz.ch/~mpt. Zürich, Switzerland, July 2013, pp. 502–510.

[3] Ehsan Lakzaei. "Balancing ball on a plate: Final Report". Final Report. Eindhoven, The Netherlands: Eindhoven University of Technology and Fontys University of Applied Sciences, July 2020.

[4] Lennart Ljung. *System Identification: Theory for the User*. 2nd. Upper Saddle River, NJ, USA: Prentice Hall, 1999. ISBN: 0-13-656695-2.

[5] PyPI. *OpenCV*. https://pypi.org/project/opencv-python/. Accessed: 2026-05-12. 2026.

[6] RealSense, Inc. *RealSense Product Family D400 Series*. RealSense, Inc. USA, 2026. URL: https://realsenseai.com/wp-content/uploads/2025/09/Intel-RealSense-D400-Series-Datasheet-October-2025.pdf.

[7] The MathWorks, Inc. *Model Predictive Control Toolbox User's Guide*. The MathWorks, Inc. Natick, MA, USA, 2024. URL: https://www.mathworks.com/products/mpc.html.

[8] Peter den Toom. "Efficient MIMO Iterative Feedback Tuning applied to a mechatronic system". Master's thesis. Eindhoven University of Technology, Mar. 2024. URL: https://research.tue.nl/en/studentTheses/efficient-mimo-iterative-feedback-tuning-applied-to-a-mechatronic/.